●開発したモデル：Semantic Segmentation：RGB画像+深度マップからピクセルごとのクラスを分類する

●工夫した点

今回のタスクの特徴として、「データセットのクラス不均衡が大きいこと」、「訓練データ数が非常に少ない状況であること」が挙げられる。これらの特徴を考慮して様々な工夫を行った。

1. backboneの工夫

今回のタスクではモデルの構造としてエンコーダー・デコーダーモデルを採用した。その際事前学習済みモデルをbackboneとして採用することで少ないデータ数でも精度を向上させるよう図った。

初めに「Resnet(Resnet18,Resnet50,Resnet101)」を試した。これらは既存のアーキテクチャに組み込みやすく、取り入れることで高い精度を出すことができたがすぐに頭打ちになってしまった。

次に「Swin Transformer」を試した。これはTransformerを何層にも重ねるにつれ入力画像の解像度を小さくしていく手法で、エンコーダーデコーダーモデルと相性が良かった。しかし事前学習に使ったデータセットと本データセットにはドメインシフトがあり、本タスクにおいてtransformerの精度を十分に出せるほどのデータセットがないと判断した。

最終的に「convnext」を採用した。これはSwin Transformerの設計思想をCNNにて代替したモデルであり、ドメンシフトがある今回のタスクにおいても帰納バイアスのおかげで良い精度が見込めると判断したため採用した。

panopatic segmentationタスクで事前学習されたswin transformerをエンコーダーに取り入れることも検討した。panopatic segumentationはsemantic segmentationと類似したタスクでドメンシフトが少ないと考えたからだ。構築した結果良い結果が出たのでアンサンブリングを行うことにした。

2. データ拡張

データ数が少ない本タスクにおいてデータ拡張は非常に重要である。本モデルにおいて次のデータ拡張を実装した。(セマンティックセグメンテーションは幾何学変換の時は訓練画像だけでなく対応するラベル画像にも同じ拡張を加える、色彩の変換は訓練画像のみに加えるという工夫が必要でありalbumentationsのライブラリを用いて実装した)

・HorizontalFlip:左右反転
・ShiftScaleRotate:移動・拡大縮小・回転
・A.RandomResizedCrop:ランダムクロップ
・A.GridDistortion、OpticalDistortion:広角レンズ特有の樽型歪みや、壁・床のパース（遠近感）の歪みの再現
・RandomBrightnessContrast / HueSaturationValue**:** 部屋の照明条件（昼夜、照明の色）の変化の再現
・CLAHE (コントラスト限定適応適応ヒストグラム等価化)**:** 暗い場所にある物体の境界の強調。
・GaussNoise / Blur / Sharpen**:** カメラのノイズ、ピンボケ、逆にエッジが立ちすぎた状態を再現。
・CoarseDropout**:** 画像内にランダムな黒い矩形（穴）をいくつか配置。
・Cut_mix:画像の一部を他の画像のパッチで置き換える(エポック後半は境界をしっかり学習するために適用しない)。

3.正則化
　本タスクではデータ数が少ないことから過学習を起こしてしまうことが課題として挙げられた。その課題を解決するために様々な正則化の手段を採用した。

・Early Stoppingの実装:本データセットの訓練データをtrain:val=9:1になるように分割し、validationデータのmIOUのスコアを基準にEarly Stoppingを実装した。
・weight_decayの強化：weightdecayを0.08ほどまで上げた。
・ドロップアウトの実装: エンコーダー・デコーダーのそれぞれでドロップアウトを実装した。

4.正規化
・訓練データを事前学習済みモデルが学習されたimagenetの統計量で正規化した。
・depth画像も訓練データのdepth画像の統計量で正規化した。

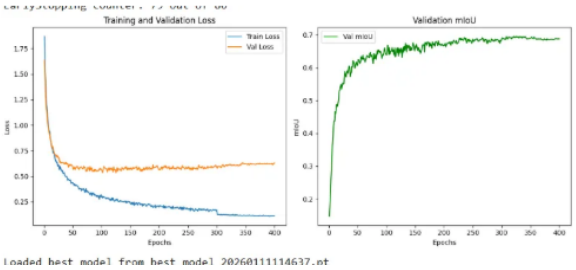
5.アーキテクチャの工夫
・初めは単純なu-net構造を用いていたがuperunet構造(エンコーダーの最終層でPyramid Pooling Module(複数の異なるビンサイズでプーリング)を用いる)を用いることで広域なコンテキストと局所的な情報を獲得できるよう図った。

6.loss関数の工夫
・dice lossという測結果と正解ラベルの「領域の重なり具合」を直接最適化するための損失関数の使用を検討した。しかしdice lossのみにすると学習が上手く進まなかったのでCross Entropyとの加重平均を取った。loss=0.8*Cross Entorpy+0.2 dice loss程度がもっとも精度が良かった。

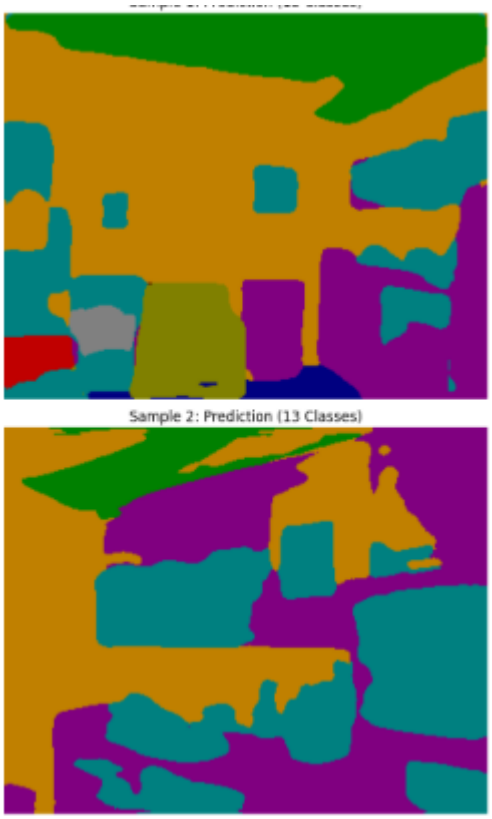
7.学習法の工夫
・optimizerの変更:AdamをAdamWに変更
・スケジューラーの設定:学習終盤で学習率を下げるスケジューラーを設定
・差分学習率:事前学習済みモデルの重みを壊さないようにエンコーダーとデコーダーで学習率を変えた。しかしエンコーダーの学習率を下げすぎると一律学習よりもむしろ精度が下がった。これはエンコーダーの入力層を4chに適応するために書き換えたことや、事前学習済みモデルとNYUv2データセットにドメンシフトが存在するからであると判断した。
よって最終的にデコーダーの0.8倍の学習率をエンコーダーに採用した。

8.視覚化による判断
・学習の損失とmIOUのスコアを可視化することで学習のたびにスコアの変動の原因を解析した。

↓例(提出notebookより)



・テストデータの出力を可視化することでタスクにおけるモデルの現状を解析した。
↓例(提出notebookより)



9.推論の工夫

・テストデータに対してTTAを行いアンサブリングの効果を図った

・convnextをエンコーダーとしたuperunetで推論した結果とswin transformerをエンコーダーとしたunetで推論した結果をアンサンブリングした。(それぞれの実行の際は最大エポック数や学習率、weight decayやエンコーダーのdrop_path_rateなどのパラメータを変更した。)

●結果:

これらの工夫を加えることでmIOUのスコアを**0.65792**まで改善することができた。

●今後試したいこと

・より強力なbackbone(Dinoなど)を使ったり、convnextとアンサンブリングしたりしてみたい。

●感想

　様々な工夫を行ったが最も効果的だったのは事前学習済みモデルのファインチューニングである。その上で採用するアーキテクチャも重要であると感じた。より大きいモデルやより賢いアーキテクチャを選択する事で精度が向上したが、その分より強力な計算資源を要する。今後の実務において自分たちが抱えるリソースを元に適切にモデルを選択したい。

　またデータの重要性を再認識した。事前学習済みモデルを用いてもドメインシフトがある場合は入力画像による学習が非常に重要である。その際少ないデータではすぐに過学習してしまい思うような精度が出せない。高い精度を出すにはデータ拡張はもちろんより多くのデータを用意する事がとても大切であるようだ。

　また検証データの重要性も認識した。私は初めのうちは少ないデータセットを考慮して全てのデータで学習を回していた。そこでlossがさがっていても良い精度にならないことが往々にしてあった。それは過学習を起こしていたからに他ならない。検証データで過学習が起きているかどうか判断することができるので検証データは必須であると感じた。

　今回の課題を通してAIモデル開発の基礎と共にそれを改良するスキルを獲得できた。今後自分がAIモデルを開発するための知識はもちろん、様々なアーキテクチャを理解する上での基礎知識も獲得できた。この経験を自身の研究や今後の実務に活かしていきたい。